Part IV

Bash scripts

Shells

- A shell is the program that interprets what you type at the command-line.
- There are many shells.
- The most common (default) is the Bourne-Again Shell (bash).
- Others include `ksh`, `csh`, `tcsh`, `zsh`.
- These can look deceptively similar, but have subtle differences.
- I shall focus on `bash`.
- Bash includes a complete programming language, with loops, if-else statements, functions, etc.
- For far more detail than you will ever need, see `https://www.tldp.org/LDP/abs/html/`

Shell script

- Instead of manually repeating a set of instructions at the command-line, you can put the commands into a single file `compile`:

```
#!/bin/bash
gcc -c main.c -o main.o
gcc -c ODE.c -o ODE.o
gcc main.o ODE.o -lm -o ODEsolver
```

- Once this is defined to be executable with `chmod u+x ./compile` you can run the command `./compile` which will run these commands.
- The first line indicates the interpreter for the file.
- You can equally use another shell or even `/usr/bin/python` or similar.

Environment variables

- There are many special environment variables that control how programs function.
- To see a full list as defined in your shell, type `export`.
- To set an existing or new environment variable for use in the whole shell, use

```
export MYNAME=pmb39
```

- This is then defined for the remainder of the shell session.
- To set a variable for the remainder of a script only, use

```
allFiles="file1.txt file2.txt"
```

- To access a variable's value, use `$MYNAME`:

```
$ echo "My name is $MYNAME"
My name is pmb39
```

This is called variable *expansion*.

Special environment variables - PATH

- `PATH` is the set of directories that are searched for executables:

  ```
  PATH=/bin:/usr/bin:.
  ```

- If a command is not found in any of these directories:

  ```
  $ castep.mpi
  castep.mpi: command not found
  ```

- so you would need to give its full path:
  `/lsc/opt/castep-18.1/castep.mpi` or put the path into `PATH`:

  ```
  export PATH=/lsc/opt/castep-18.1:$PATH
  ```

- The directories are searched in order from the beginning, and the first one to contain the required executable is used.

  ```
  echo $PATH
  /lsc/opt/bin:/usr/local/cuda/bin:/usr/local/sbin:
  /usr/local/bin:/usr/bin:/bin
  ```

Making use of PATH

- It can be useful to create a directory `~/bin` and add this to your `PATH`:
  `export PATH=~/bin:$PATH`
- Then, you can make various useful scripts available everywhere.
- Note the existence of the `which` command:

```
$ which g++
/usr/bin/g++
$ which spellCheck
/home/pmb39/bin/spellCheck
```

- You can add extra lines into the file `~/.bashrc` to automatically add this directory for every new shell/terminal you start.

Special environment variables

- `SHELL` - the current shell e.g. `/bin/bash`
- `HOSTNAME` - the current computer name e.g. `csc23-laptop-25`
- `PWD` - the current working directory
- `OLDPWD` - the previous working directory - use `cd -` to go there.
- `HOME` - your home directory on this computer - use `cd` to go there

Regular Expressions

- We may wish to operate on a set of files whose names conform to a particular format:
- `ls *.C` will list all files ending in ".C"
- `ls *[0-9].txt` will list all files ending in a digit followed by `.txt`
- `ls Water*[02468].txt` will list all files starting with `Water` and ending with an even digit followed by `.txt`.
- Strictly, Bash uses the POSIX Extended Regular Expression (ERE) dialect of regular expressions.

Basic quoting and escaping

- To preserve the literal meaning of characters, enclose them in " "
- So `ls "*.C"` will list all files called, literally, `*.C`, of which there will probably be none.
- `ls "My Documents"` will list the directory called `My Documents`.
- `ls My Documents` will try to list the files/directories called `My` and `Documents`.
- Or, `ls My\ Documents` will list the directory `My Documents`
- The backslash "escapes" the following space, i.e. interprets it as an ordinary character, rather than as a word separator.
- You are strongly advised to avoid having spaces in directory names, to avoid problems in scripts and commands that fail to deal with them correctly. I use _ instead.

Command substitution

- If you want the result of one command to be used later on, you can use command substitution:

```
files=$(find -name "*.pdf")
```

- The command-expansion can contain piped or other commands:

```
words=$(detex Thesis.tex | spell | sort | uniq)
```

- You may also see an older form using backticks ‘ ‘:

```
words=`detex Thesis.tex | spell | sort | uniq`
```

  This works, but can be more fragile if you need to nest commands or escape the backtick.

Exit codes

- All Linux programs return an integer between 0 and 255.
- C/C++ programmers will recognise this from `int main(void)`
- Usually, 0 indicates the program exited correctly with no errors.
- Other exit codes depend on the application, and should be listed in the `man` page.
- For example, for `grep`:

```
EXIT STATUS
       The exit status is 0 if selected lines are found,
       and 1 if not found.
       If an error occurred the exit status is 2.
```

Special variables

There are variables whose value changes depending on the current shell script, etc.

Variable	Description	Example
`$?`	Exit code of last process run	0
`$#`	Number of command-line arguments passed to a script	2
`$*`	All command-line parameters, as a single word	`myFile.txt myFile2.txt`
`$@`	As `$*` but with words separately quoted	`myFile.txt myFile2.txt`
`$0`, `$1`, `$2` etc.	Successive command-line parameters passed to script, starting with command-name	`ls`, `./`

Conditional statements - test

- In `bash`, the `test` command is capable of comparing strings, integers, and testing whether files exist.
- For example:

```
if test $? -ne 0; then
  echo "Error"
  exit 1
fi
```

  will only print "Error" (and exit the script) if the previous command exited with exit code not equal to zero.

- Other options include file-existence:

```
if test -f output.txt; then
  echo "output.txt exists. Will not overwrite."
  exit 1
fi
```

- See `man test` for more details.

Logical operations

- `test` can deal with AND (`-a`) and OR (`-o`).
- So can the shell, with `&&` and `||`
- These employ short-circuiting, i.e. work from left to right and stop as soon as the result is known.
- Here 0 is true (success), and anything else is false (error).

  ```
  mkdir Pictures && mv *.png ./Pictures
  ```

  will only move files if the directory has been successfully made.
- If there had already been a *file* called `Pictures`, we might otherwise have overwritten it.

Extended test

- You will also see the `[[ ... ]]` syntax instead of `test`.
- For example:

```
if [[ $# -ge 2 && -f "$1" && -f "$2" ]]; then
...
fi
```

- If we want to check that the script has been passed at least 2 arguments and that both are files.
- Note the spaces inside `[[ ]]` separating the brackets from their contents.

Looping - for

- There are two main forms of `for` loop in bash:
  `for f in $myFiles; do cp $f $f.bak; done`
  would copy all files given in the `myFiles` variable to backup versions of same.

```
for ((i=0 ; i < 10 ; i++)); do
  mv "Data$i.txt" ./FirstPass/
done
```

  would move 10 files into the directory `FirstPass`.
- You can write everything on one line with ;s or on multiple lines, either in a script or at the command line.
- Braces may be needed around the variable being expanded:

```
mv "Data_$i_coarse.txt" ./FirstPass/
mv "Data_${i}_coarse.txt" ./FirstPass/
```

- In the first version the shell would attempt to expand the variable `i_coarse`, resulting in `Data_.txt`

Looping - while

- There is also the do-while loop:

```
while true; do
  echo "Hello"
done
```

  would print “Hello” for ever.

- Use `break` to exit a loop early.
- Use `continue` to go immediately to the next iteration of a loop, for example from inside an `if` statement.

Reading user input - read

- You may want user input during a shell script.
- `read a b` will read two words from the user into variables `a` and `b`
- Then to read repeatedly from stdin:

```
while true; do
  read a b || break
  echo "Received pair of inputs ${a} and ${b}"
done
```

- If `read` fails (i.e. no input left), then it will return exit-code 1, corresponding to failure/false, therefore `break` will be evaluated, so that execution of the loop stops.

Bash functions

You can write functions in Bash:

```
function create {
  if [[ -f $1 ]]; then
    echo "File $1 already exists
  fi
  if [[ -d $1 ]]; then
    echo "Directory $1 already exists
  fi
  touch $1
  if [[ "$?" -ne 0 ]]; then
    echo "Error in creating file $1"
  fi
}

create README
```

Debugging scripts - shellchec

- Writing robust shell-scripts is hard.
- Common mistakes result in spaces in file-names not being handled correctly.
- The **shellcheck** command can help with this.
- It makes suggestions on possible errors in your scripts.

```
$ shellcheck test_makefile.sh
In test_makefile.sh line 3:
rm -f example1 example2 *.o
                        ^-- SC2035: Use ./*glob* or -- *glob*
                    so names with dashes won't become options.
In test_makefile.sh line 59:
commands=`make all |& wc -l`
         ^-----------------^ SC2006: Use $(...) notation
         instead of legacy backticked `...`.
```

Extended #!

- Earlier, you may have wondered about the `#!/bin/bash` at the start of a bash-script.
- The #! (hash-bang) are special characters indicating that the next thing on the line is a separate executable that will be used to parse the file.
- This could be any parser, such as:

```
#! /usr/bin/sed -f
#! /usr/bin/python
...
```

  Note the `-f` for `sed` because what actually happens is that the filename is appended to the `#!` line, and `sed` requires the `-f` flag in this case.

- (If you start writing `sed` scripts into a file, stop and reconsider your life-choices.)